ESTACIA

ESTACIA
Arquitetura de Software
Versão <1.3>

Histórico da Revisão

Data	Versão	Descrição	Autor
14/05/26	1.0	Elaboração do documento	Larissa Vargas Moreira
03/06/26	1.1	Acréscimo do diagrama de pacotes e metodologia	Larissa Vargas Moreira
17/09/26	1.2	Mudança na arquitetura das rotas (Troca da API do Google maps pelo IPS) e definição do modelo de IA	Letícia Araújo da Silva Amparo
22/09/26	1.3	Atualização dos entregáveis nas sprints e do caso de uso	Letícia Araújo da Silva Amparo

Sumário

1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 4
1.4 Visão Geral 5
2. Metodologia Utilizada 6
3. Representação Arquitetural 9
4. Metas e Restrições da Arquitetura 10
5. Visão de Casos de Uso 11
6. Visão Lógica 13
6.1 Visão Geral e Pacote de Design 13
6.2 Especificação da Navegação Indoor (IPS) e Grafos 13
6.3 Estrutura do Banco de Dados 14
7. Visão de Implantação 15
8. Visão da Implementação e Motor de Inteligência Artificial 16
9. Tamanho e Desempenho 17
9.1 Qualidade 17

1. Introdução

O presente documento tem como objetivo descrever o documento de arquitetura do software EstacIA. Sua finalidade é contextualizar a motivação para o desenvolvimento do projeto, definindo o propósito fundamental, organização estrutural e diretrizes técnicas para suporte à equipe de desenvolvimento e acompanhamento de stakeholders.

1.1 Finalidade

Este documento apresenta uma visão geral da arquitetura do sistema EstacIA, um gerenciador inteligente de garagens para centros comerciais (shoppings). O objetivo principal é capturar e comunicar as decisões arquiteturais significativas tomadas para o projeto, servindo como guia técnico para a equipe de desenvolvimento e garantindo que o sistema atenda aos critérios de escalabilidade, desempenho e integração requeridos. O público-alvo deste documento inclui os desenvolvedores do sistema, a equipe de garantia de qualidade, os gestores do projeto e os stakeholder do centro comercial interessados em compreender a viabilidade técnica e a estrutura de funcionamento do projeto.

1.2 Escopo

O EstacIA consiste em uma solução de software voltada para o direcionamento inteligente e gerenciamento de veículos em pátios de estacionamento de grandes centros comerciais (shoppings). O escopo da solução engloba o desenvolvimento de um aplicativo móvel interativa (Mobile-First / PWA) focado na jornada do condutor desde a localização preditiva de vagas livres até o registro autônomo por QR Code, cálculo de rota indoor da vaga por inteligência artificial, localização e direcionamento do carro ("Encontre Meu Carro") e liquidação financeira do tempo de permanência. Este documento auxilia diretamente todos os envolvidos no projeto a compreenderem os aspectos arquiteturais necessários para a consolidação e construção dessa plataforma.

1.3 Definições, Acrônimos e Abreviações

- **API (Application Programming Interface):** Interface de Programação de Aplicação utilizada para comunicação entre cliente, servidor e dispositivos IoT.
- **QR Code (Quick Response Code):** Código de barras bidimensional utilizado no EstacIA para o registro autônomo da vaga e check-in.
- **MVC (Model-View-Controller):** Padrão arquitetural que divide a aplicação em três camadas lógicas interconectadas.
- **IPS (Indoor Positioning System):** Sistema de posicionamento e navegação em ambientes fechados onde o GPS via satélite não possui precisão ou alcance.
- **Algoritmo A* (A-Star):** Algoritmo de busca heurística em grafos utilizado para calcular a menor rota indoor considerando múltiplos andares.

- **YOLOv8 / Visão Computacional:** Modelo de inteligência artificial convolucional aplicado na extração e conversão automatizada de plantas baixas em grafos.
- **XGBoost / Random Forest:** Algoritmo de aprendizado de máquina supervisionado para recomendação preditiva de vagas.
- **IoT (Internet of Things):** Sensores de presença instalados nas vagas para monitoramento em tempo real.
- **Tolerância:** Período inicial de 20 minutos de permanência gratuita no estacionamento.

1.4 Visão Geral

Este documento está organizado em seções modulares cobrindo a metodologia Scrumban, a representação arquitetural multinível, os casos de uso, a visão lógica detalhada com suporte a grafos $3D$, o motor de IA e a visão da implementação tecnológica.

2. Metodologia Utilizada

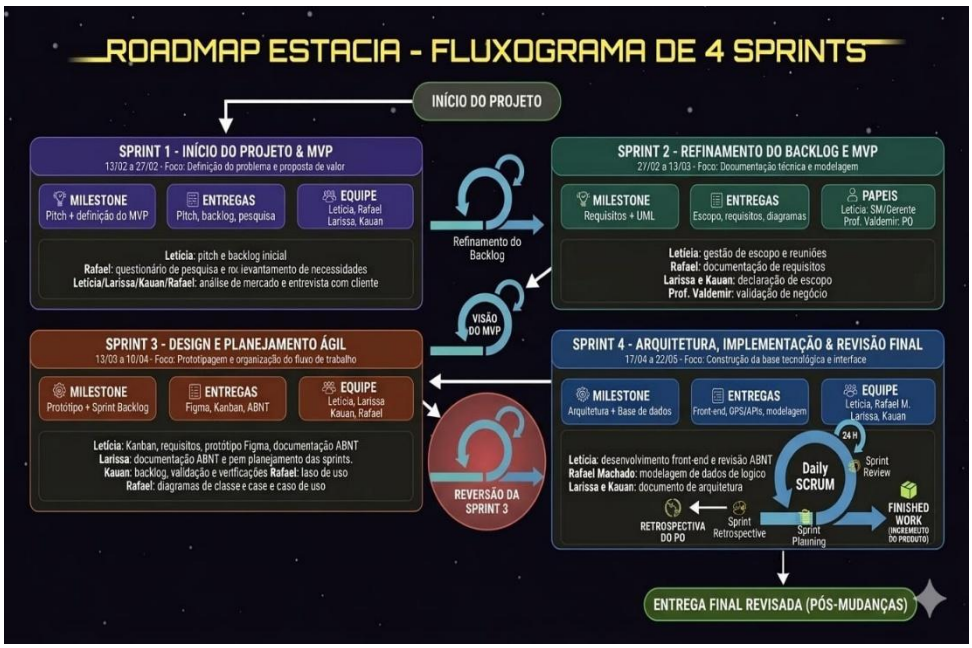
Para o desenvolvimento do sistema Estacia foi adotada a metodologia ágil Scrumban, que combina práticas do Scrum e do Kanban. Essa abordagem foi escolhida por proporcionar flexibilidade no gerenciamento das atividades, aliando planejamento estruturado em sprints com acompanhamento visual do fluxo de tarefas.

O desenvolvimento do sistema foi organizado em sprints, nas quais são planejadas e executadas funcionalidades específicas. No início de cada ciclo, a equipe define as atividades prioritárias do backlog, estabelecendo metas e entregas para o período.

As tarefas são organizadas em um quadro Scrumban composto pelas etapas: Backlog, A Fazer, Em Desenvolvimento, Em Teste e Concluído. À medida que são executadas, as atividades são movimentadas entre as etapas, permitindo o acompanhamento do progresso.

A equipe atua de forma colaborativa, porém com papeis definidos, sendo eles: scrum master e gerente de projeto (Letícia) responsável pelo planejamento, gerencia e garantia da metodologia e desenvolvedores (Letícia, Larissa, Rafael e Kauan) sendo responsáveis pelo desenvolvimento, testes e documentação do sistema, garantindo organização e alinhamento.

A adoção dessa metodologia contribui para maior controle das atividades, flexibilidade na evolução do sistema e acompanhamento contínuo das entregas, favorecendo o desenvolvimento estruturado do projeto.



Sprint	Foco Principal	Milestone e Entregas
Sprint 1	Início do Projeto & MVP	Pitch, definição do problema, proposta de valor e backlog inicial.
Sprint 2	Refinamento e Modelagem	Levantamento de requisitos, diagramas UML e declaração de escopo.
Sprint 3	Design e Planejamento Ágil	Protótipo no Figma, quadro Kanban e validação das regras de negócio.
Sprint 4	Arquitetura	Documento de arquitetura e banco de dados.
Sprint 5	Definição da IA, rota e implementação.	Motor de IA indoor, sensores e integração no banco de dados.
Sprint 6	Desenvolvimento e treinamento da IA e mapeamento	Algoritmo de conversão de imagem/planta baixa para Garfo de Rotas (A*)
Sprint 7	Backend	Integrar consulta da API de vagas com a rota dinâmica + conectar a tela “Encontre meu carro” com o endpoint da IA para desenhar a linha do mapa dinamicamente com base na resposta da API

Sprint	Foco Principal	Milestone e Entregas
Sprint 8	Testes	Refinamento do modelo, testes e correção de erros

3. Representação Arquitetural

O projeto adota o padrão arquitetural MVC (Model-View-Controller) integrado ao modelo Cliente-Servidor. Essa abordagem foi escolhida para garantir a separação de responsabilidades, facilitando assim a portabilidade do aplicativo móvel e a escalabilidade dos serviços centrais. A estrutura arquitetural ficou organizada através das seguintes camadas:

- **Camada de apresentação (cliente):** Composta pelas interfaces do usuário, englobando o aplicativo móvel utilizado pelos motoristas para navegação e leitura de QR Code. Renderizada vetorialmente os mapas interativos SVG/Canvas.
- **Camada de controle e negócio (Servidor Backend em Python/Django):** Centraliza as regras de negócio (cálculo de tempo de tolerância de 20 minutos, precificação), o motor de rotas indoor A^* e o serviço de recomendação preditiva. Vai agir como intermediária entre as requisições dos clientes e o armazenamento.
- **Camada de Inteligência Artificial e Ingestão IoT:** Processa os dados de telemetria dos sensores ultrassônicos das vagas e executa a conversão de plantas baixas via visão computacional.
- **Camada de dados (Banco de dados):** Responsável pela persistência e integridade das informações transacionais, armazenando a malha topológica do grafo $3D$ (nós, arestas, andares) e mapeando entidades essenciais como o status de ocupação das vagas e setores.

4. Metas e Restrições da Arquitetura

As principais diretrizes que moldam a arquitetura do Estacia compreendem:

- **Desempenho:** O processamento da leitura do QR Code e a atualização de status da vaga no banco de dados devem ocorrer em um tempo inferior a 2 segundos para evitar retenção de veículos nas cancelas.
- **Navegação Indoor Autônoma:** O sistema não utiliza a API do Google Maps para áreas cobertas ou subsolos, devendo processar o caminho internamente via algoritmo A* com precisão de andares.
- **Disponibilidade:** O servidor backend e o banco de dados devem operar com alta disponibilidade (24/7), garantindo o funcionamento ininterrupto durante todo o horário operacional do shopping.
- **Segurança:** Toda a transmissão de dados transacionais dos veículos e informações cadastrais dos usuários deve ser criptografada via protocolo HTTPS.
- **Restrições Técnicas:** O aplicativo mobile depende obrigatoriamente de conexão ativa com a internet (seja via dados móveis ou Wi-Fi fornecido pelo shopping) para buscar o grafo atualizado e transmitir os eventos do motorista.

5. Visão de Casos de Uso

A arquitetura suporta os seguintes fluxos principais do motorista e da administração:

- **Efetuar Login / Cadastro:** Acesso do usuário ao aplicativo.
- **Consultar Vagas Disponíveis:** Visualização em tempo real das vagas e setores com sugestão inteligente de vaga por IA.
- **Gerar Rota Indoor Inteligente:** Traço e exibição da rota a pé ou de carro até o setor/vaga ou de volta ao veículo ocupado ("Encontre Meu Carro"), sem dependência de serviços externos.
- **Registrar Estacionamento (Check-in):** Leitura de QR Code na coluna da vaga ou recepção de sinal do sensor IoT acoplado à vaga.
- **Realizar Pagamento Digital:** Liquidação de valores de permanência via Pix/Cartão através de Gateway seguro.
- **Consultar Histórico de Estadias:** Acompanhamento de acessos passados e recibos.

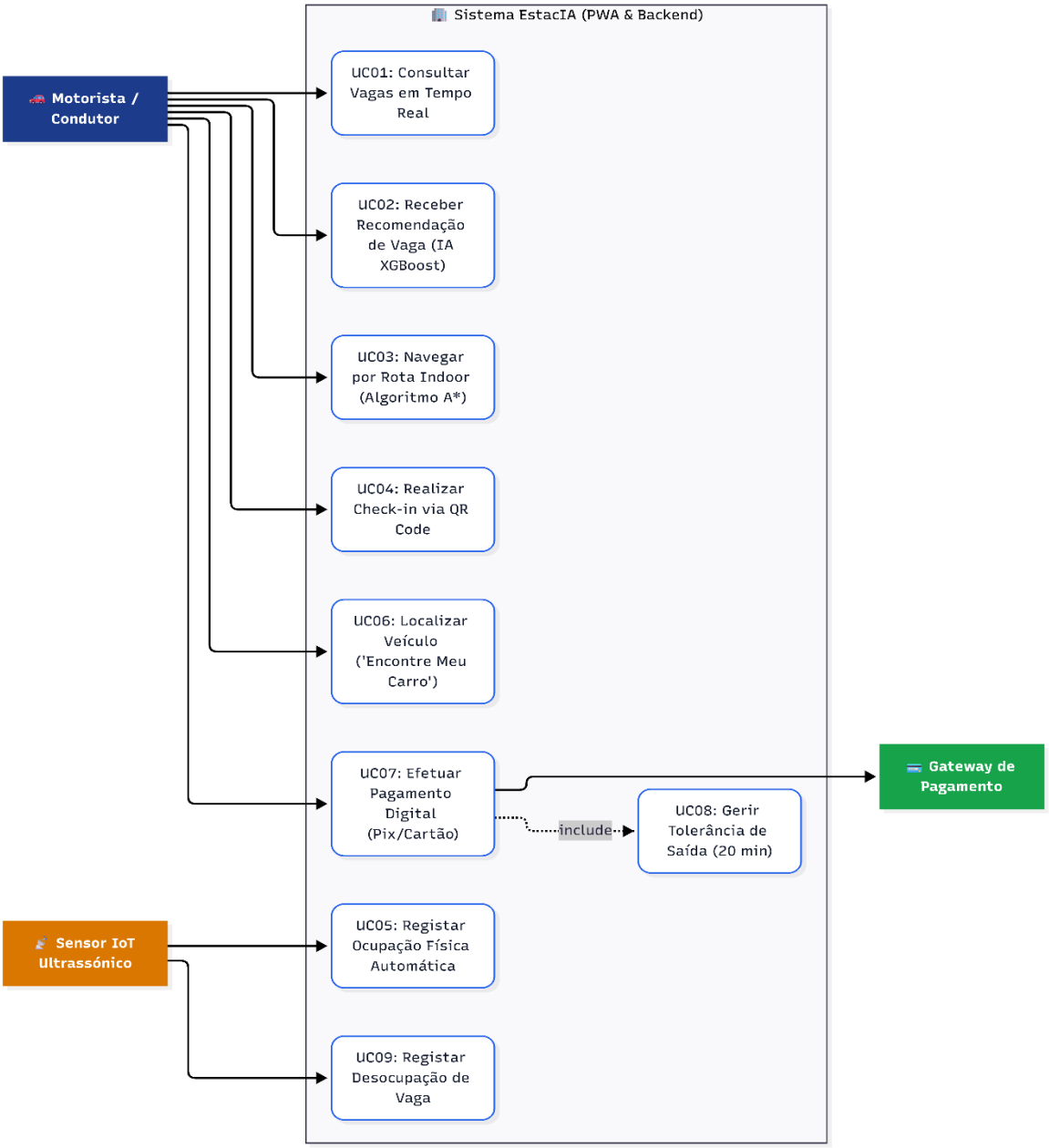


Diagrama de Caso de Uso.

6. Visão Lógica

6.1 Visão Geral e Pacote de Design

A visão lógica define a estrutura da arquitetura. Abaixo será especificado o padrão utilizado para o desenvolvimento do sistema, no caso, MVC.

- **View (Apresentação)**: Composta pelas interfaces mobile do motorista (telas de mapa, scanner de QR Code).
- **Controller (Controle)**: Mapeamento de endpoints API REST e tratamento de requisições.
- **BusinessLogic / AIServices (Lógica de Negócio e IA)**: Regras de tolerância, faturamento, motor A^* e modelo preditivo.
- **Persistence (Persistência):** ORM e comunicação direta com o banco de dados SQL. (inserção de tempo de estadia, atualização do status da vaga), mapeando as entidades do sistema.

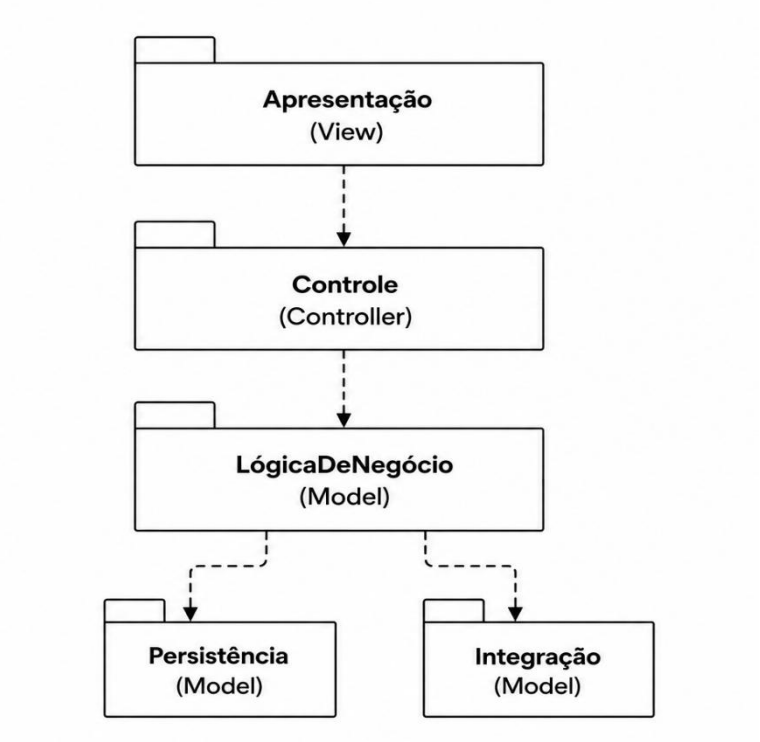


Diagrama de Pacotes.

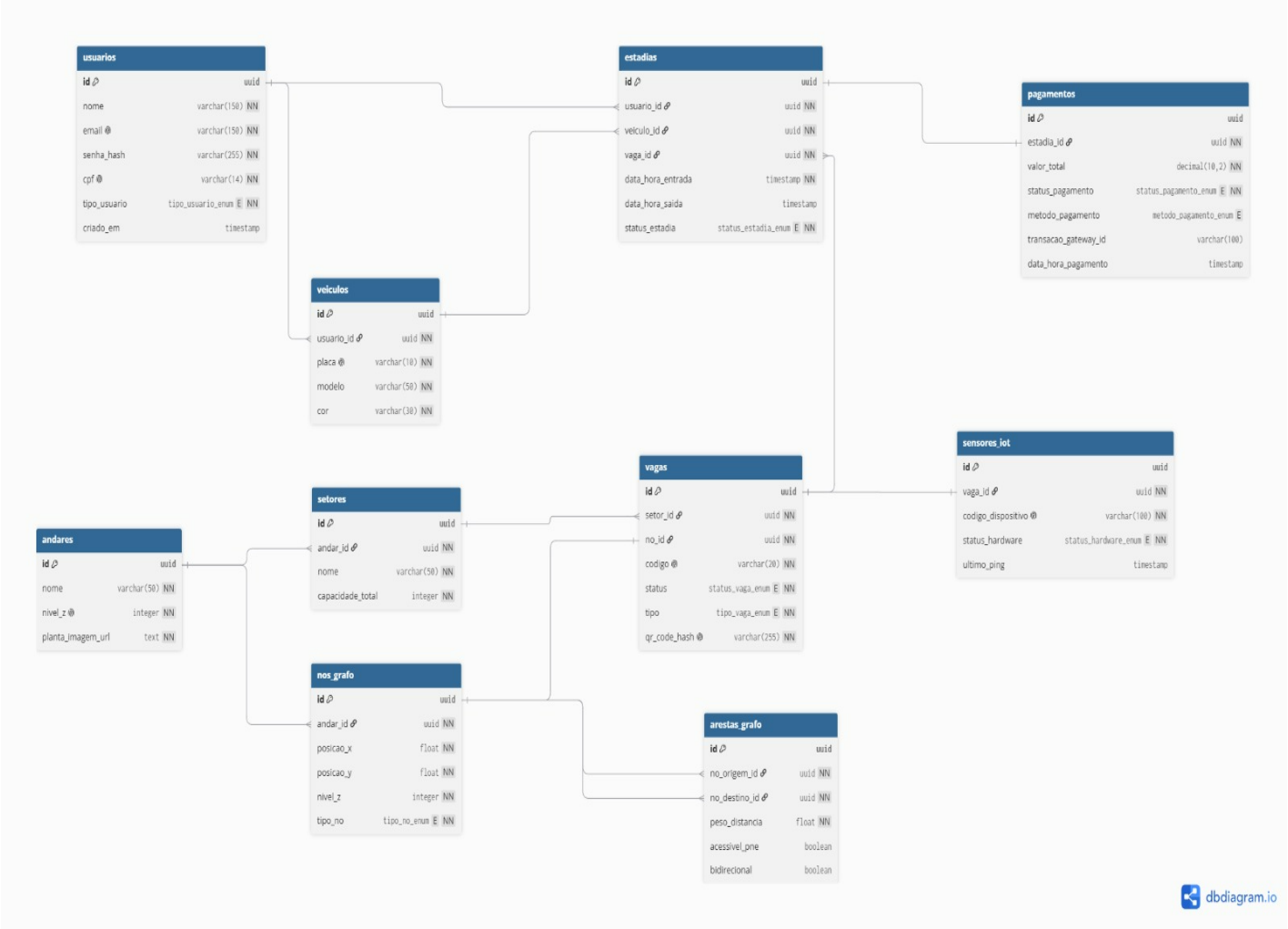
6.2 Especificação da Navegação Indoor (IPS) e Grafos

Devido ao bloqueio de sinais GPS em subsolos e estruturas metálicas, o EstacIA implementa um modelo de navegação topológico por grafo $3D$:

1. **Grafo Topológico:** A planta é representada como $G = (V, E)$, onde cada vértice V possui coordenadas (X, Y, Z) e cada aresta E possui pesos de distância e acessibilidade.
   - X, Y: Posição cartesiana na imagem do andar.
   - Z: Nível do andar (ex: 0 para Térreo, -1 para Subsolo 1).

2. **Nós de Transição:** Pontos especiais do tipo 'ELEVADOR', 'ESCADA_ROLANTE' e 'RAMPA' conectam diferentes níveis Z, permitindo cálculo de rotas multinível

6.3 Estrutura do Banco de Dados



Modelo Físico do Dados

7. Visão de Implantação

O ecossistema é implantado em arquitetura Cloud-Native:

- **Servidor de Aplicação & Backend IA:** Hospedado em containers Docker na plataforma Render / AWS, rodando serviços Python/Django e FastAPI.
- **Banco de Dados:** PostgreSQL gerenciado no Supabase com suporte a rotinas otimizadas e índices espaciais.
- **Frontends:** Aplicativo distribuído via PWA/Stores e Painel Web servido via CDN.
- **Dispositivos Físicos:** Sensores IoT conectados via protocolo MQTT aos brokers de ingestão do backend.

8. Visão da Implementação e Motor de Inteligência Artificial

A implementação do backend é fundamentada em Python/Django e Programação Orientada a Objetos (POO). Em substituição integral a serviços de mapas de terceiros (como Google Maps API), o sistema executa um Motor de IA interno baseado em 3 pilares:

1. **Processamento de Planta Baixa (YOLOv8 + OCR / OpenCV):** As plantas em PDF/imagem enviadas pelo shopping são processadas por visão computacional para reconhecer paredes, corredores, elevadores e placas de lojas, gerando automaticamente a tabela de nós e arestas do grafo.
2. **Algoritmo de Rota Indoor (A^* - A-Star):** Executa a busca pelo menor caminho no grafo tridimensional. Quando a rota exige mudança de nível ($Z_0 \neq Z_1$), o algoritmo força a transição por um nó do tipo elevador ou escada, retornando o vetor de coordenadas $[X, Y, Z]$ para desenho dinâmico na tela do usuário.
3. **Engine Preditiva de Vagas (XGBoost):** Analisa a telemetria dos sensores IoT em tempo real, os horários de pico e o destino do motorista para sugerir a vaga ideal com menor tempo de caminhada.

9. Tamanho e Desempenho

O dimensionamento estrutural e os critérios de performance do ecossistema EstacIA foram projetados para suportar com eficiência as características operacionais e os picos de fluxo típicos de grandes pátios de centros comerciais. A arquitetura desenvolvida em Python e Django precisa ser otimizada para processar múltiplos volumes de acessos simultâneos, cenários que ocorrem com frequência em horários de almoço, finais de semana e períodos sazonais de compras. Para mitigar o risco de formação de filas e gargalos operacionais nas cancelas, o tempo total estimado de resposta de processamento — compreendendo o instante em que a câmera do smartphone lê o QR Code, o envio desse dado ao servidor backend, a validação das regras de negócio e o retorno de liberação — deve ocorrer em um intervalo inferior a dois segundos. Adicionalmente, as consultas realizadas na base de dados para atualizar e contabilizar a disponibilidade das vagas em tempo real devem utilizar rotinas de buscas otimizadas e indexações bem estruturadas, impedindo que requisições concorrentes gerem travamentos de tabelas ou lentidão generalizada no sistema.

9.1 Qualidade

Os atributos de qualidade do software são assegurados diretamente pelas decisões de design documentadas nesta arquitetura, destacando-se as metas de alta manutenibilidade e portabilidade da solução. A divisão transparente de responsabilidades lógicas entre a interface visual e o servidor garante que o código-fonte seja modular, permitindo que futuras manutenções sejam ágeis e seguras. Dessa forma, caso a administração do shopping decida alterar parâmetros de negócio no futuro, como estender a janela regulamentar de tolerância para além dos 20 minutos gratuitos ou modificar as políticas de faturamento, essas alterações ficarão restritas estritamente à camada lógica interna, sem qualquer necessidade de refatorar as telas do aplicativo móvel ou corromper a integridade das tabelas armazenadas. Além disso, a modelagem lógica estruturada inicialmente no ambiente de design do *brModelo* e sua respectiva tradução física para a linguagem estruturada SQL garantem a portabilidade dos dados, assegurando que o sistema permaneça robusto, de fácil leitura para novos desenvolvedores e preparado para futuras expansões de infraestrutura.